Control flow, list comprehensions, and functions

Lecture 03

Dr. Colin Rundel

Control Flow

Conditionals

Python supports typical if / else style conditional expressions,

```python
x = 42

if x < 0:
    print("X is negative")
elif x > 0:
    print("X is positive")
else:
    print("X is zero")
```

```
X is positive
```

```python
x = 0

if x < 0:
    print("X is negative")
elif x > 0:
    print("X is positive")
else:
    print("X is zero")
```

```
X is zero
```

Significant whitespace

This is a fairly unique feature of Python - expressions are grouped together via *indenting*. This is relevant for control flow (`if`, `for`, `while`, etc.) as well as function and class definitions and many other aspects of the language.

Indents should be 2 or more spaces (4 is generally preferred based on PEP 8) or tab character(s) - generally your IDE will handle this for you.

If there are not multiple expressions then indenting is optional, e.g.

```
if x == 0: print("X is zero")
```

```
X is zero
```

Conditional scope

Conditional expressions do not have their own scope, so variables defined within will be accessible / modified outside of the conditional.

This is also true for other control flow constructs (e.g. `for`, `while`, etc.)

```
s = 0
s
```

```
0
```

```
if True:
    s = 3
s
```

```
3
```

`while` loops

repeats until the condition expression evaluates to `False`,

```
i = 17
seq = [i]

while i != 1:
    if i % 2 == 0:
        i /= 2
    else:
        i = 3*i + 1

    seq.append(i)

seq
```

```
[17, 52, 26.0, 13.0, 40.0, 20.0, 10.0, 5.0, 16.0, 8.0, 4.0, 2.0,
```

for loops

iterates over the elements of a *sequence*,

```
for w in ["Hello", "world!"]:
    print(w, ":", len(w))
```

```
Hello : 5
world! : 6
```

```
sum = 0
for v in (1,2,3,4):
    sum += v
sum
```

```
10
```

```
res = []
for c in "abc123def567":
    if (c.isnumeric()):
        res.append(int(c))
res
```

```
[1, 2, 3, 5, 6, 7]
```

```
res = []
for i in range(0,10):
    res += [i]
res
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

`break` and `continue`

allow for either an early loop exit or step to the next iteration respectively,

```
for i in range(1,10):
    if i % 3 == 0:
        continue

    print(i, end=" ")
```

```
1 2 4 5 7 8
```

```
for i in range(1,10):
    if i % 3 == 0:
        break

    print(i, end=" ")
```

```
1 2
```

loops and `else`

Both `for` and `while` loops can also include an `else` clause which executes when the loop completes by either fully iterating (`for`) or meeting the `while` condition, i.e. when `break` is not used.

```
for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            print(n, 'equals', x, '*', n//x)
            break
    else:
        print(n, 'is a prime number')
```

```
2 is a prime number
3 is a prime number
4 equals 2 * 2
5 is a prime number
6 equals 2 * 3
7 is a prime number
8 equals 2 * 4
9 equals 3 * 3
```

pass

is a placeholder expression that does nothing, it can be used when an expression is needed syntactically.

```
x = -3
if x < 0:
    pass
elif x % 2 == 0:
    print("x is even")
elif x % 2 == 1:
    print("x is odd")
```

List comprehensions

Basics

List comprehensions provide a concise syntax for generating lists (or other sequences) via iteration over another list (or sequence).

```
res = []
for x in range(10):
    res.append(x**2)
res
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
[x**2 for x in range(10)]
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Since it uses the for loop syntax, any sequence / iterable object is fair game:

```
[x**2 for x in [1,2,3]]
```

```
[1, 4, 9]
```

```
[x**2 for x in (1,2,3)]
```

```
[1, 4, 9]
```

```
[c.lower() for c in "Hello World!"]
```

```
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd', '!']
```

Using `if`

List comprehensions can include a conditional clause(s) to filter the input list / sequence,

```
[x**2 for x in range(10) if x % 2 == 0]
```

```
[0, 4, 16, 36, 64]
```

```
[x**2 for x in range(10) if x % 2 == 1]
```

```
[1, 9, 25, 49, 81]
```

The comprehension can include multiple `if` statements (equivalent to using `and`)

```
[x**2 for x in range(10) if x % 2 == 0 if x % 3 ==0]
```

```
[0, 36]
```

```
[x**2 for x in range(10) if x % 2 == 0 and x % 3 ==0]
```

```
[0, 36]
```

Multiple `for` keywords

Similarly, the comprehension can also contain multiple `for` statements which are equivalent to nested `for` loops,

```
res = []
for x in range(3):
    for y in range(3):
        res.append((x,y))
res
```

```
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1),
```

```
[(x, y) for x in range(3) for y in range(3)]
```

```
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1),
```

zip

`zip` is a useful function for "joining" the elements of multiple sequences (so they can be jointly iterated over),

```
x = [1,2,3]
y = [3,2,1]
z = zip(x, y)
z
```

```
<zip object at 0x10e6fbac0>
```

```
list(z)
```

```
[(1, 3), (2, 2), (3, 1)]
```

```
[a**b for a,b in zip(x,y)]
```

```
[1, 4, 3]
```

```
[b**a for a,b in zip(x,y)]
```

```
[3, 4, 1]
```

`zip` and length mismatches

The length of the shortest sequence will be used, additional elements will be ignored (silently)

```
x = [1,2,3,4]
y = range(3)
z = "ABCDE"
```

```
list(zip(x,y))
```

```
[(1, 0), (2, 1), (3, 2)]
```

```
list(zip(x,z))
```

```
[(1, 'A'), (2, 'B'), (3, 'C'), (4, 'D')]
```

```
list(zip(x,y,z))
```

```
[(1, 0, 'A'), (2, 1, 'B'), (3, 2, 'C')]
```

Exercise 1

Using list comprehensions, complete the following tasks:

- Create a list containing tuples of x and y coordinates of all points of a regular grid for $x \in [0, 10]$ and $y \in [0, 10]$.
- Count the number of points where $y > x$.
- Count the number of points where x or y is prime.

Functions

Basic functions

Functions are defined using `def`, arguments can optionally have default values. (Arguments with defaults *must* must follow the arguments without defaults)

```
def f(x, y=2, z=3):
    print(f"x={x}, y={y}, z={z}")
```

```
f(1)
```

```
x=1, y=2, z=3
```

```
f(1,z=-1)
```

```
x=1, y=2, z=-1
```

```
f("abc", y=True)
```

```
x=abc, y=True, z=3
```

```
f(z=-1, x=0)
```

```
x=0, y=2, z=-1
```

```
f()
```

```
TypeError: f() missing 1 require
```

`return` statements

Functions must explicitly include a `return` statement to return a value.

```
def f(x):
    x**2

f(2)
type(f(2))
```

```
<class 'NoneType'>
```

```
def g(x):
    return x**2
g(2)
```

```
4
```

```
type(g(2))
```

```
<class 'int'>
```

Functions can contain multiple `return` statements

```
def is_odd(x):
    if x % 2 == 0: return False
    else:          return True
```

```
is_odd(2)
```

```
False
```

```
is_odd(3)
```

```
True
```

Multiple return values

Functions can return multiple values using a tuple or list,

```
def f():
    return (1,2,3)
f()
```

```
(1, 2, 3)
```

```
def g():
    return [1,2,3]
g()
```

```
[1, 2, 3]
```

If multiple values are present and not in a sequence, then it will default to a tuple,

```
def h():
    return 1,2,3
h()
```

```
(1, 2, 3)
```

```
def i():
    return 1, [2, 3]
i()
```

```
(1, [2, 3])
```

Docstrings

A common practice in Python is to document functions (and other objects) using a doc string - this is a short concise summary of the object's purpose. Docstrings are specified by supplying a string as the first line in the function definition.

```python
def f():
    "Hello, I am the function f() \
and I don't do anything"

    pass

f.__doc__
```

```
"Hello, I am the function f() and I don't
```

```python
def g():
    """This function also
does absolutely nothing.
"""

    pass

g.__doc__
```

```
'This function also \ndoes absolutely noth
```

Using docstrings

```
print(max.__doc__)
```

```
max(iterable, *[, default=obj, key=func]) -> value
max(arg1, arg2, *args, *[, key=func]) -> value

With a single iterable argument, return its biggest item. The
default keyword-only argument specifies an object to return if
the provided iterable is empty.
With two or more arguments, return the largest argument.
```

```
print(str.__doc__)
```

```
str(object='') -> str
str(bytes_or_buffer[, encoding[, errors]]) -> str

Create a new string object from the given object. If encoding or
errors is specified, then the object must expose a data buffer
that will be decoded using the given encoding and error handler.
Otherwise, returns the result of object.__str__() (if defined)
or repr(object).
encoding defaults to sys.getdefaultencoding().
errors defaults to 'strict'.
```

```
print("".lower.__doc__)
```

```
Return a copy of the string converted to lowercase.
```

Argument order

In Python the argument order matters - positional arguments must always come before keyword arguments.

```
def f(x, y, z):
    print(f"x={x}, y={y}, z={z}")
```

```
f(1,2,3)
```

```
x=1, y=2, z=3
```

```
f(x=1,y=2,z=3)
```

```
x=1, y=2, z=3
```

```
f(1,y=2,z=3)
```

```
x=1, y=2, z=3
```

```
f(y=2,x=1,z=3)
```

```
x=1, y=2, z=3
```

```
f(x=1,y=2,3)
```

```
positional argument follows keyword argume
```

```
f(x=1,2,z=3)
```

```
positional argument follows keyword argume
```

```
f(1,2,z=3)
```

```
x=1, y=2, z=3
```

Positional vs keyword arguments

```
def f(pos1, pos2, /, pos_or_kwd, *, kwd1, kwd2):
      -----------    ----------     ----------
        |             |                   |
        |        Positional or keyword    |
        |                                 - Keyword only
         -- Positional only
```

For the following function x can only be passed by position and z only by name

```python
def f(x, /, y, *, z):
    print(f"x={x}, y={y}, z={z}")
```

```python
f(1,1,z=1)
```

```
x=1, y=1, z=1
```

```python
f(1,y=1,z=1)
```

```
x=1, y=1, z=1
```

```python
f(1,1,1)
```

```
TypeError: f() takes 2 positional arguments but 3 were given
```

```python
f(x=1,y=1,z=1)
```

```
TypeError: f() got some positional-only arguments passed as keyword arguments: 'x'
```

Variadic arguments

If the number of arguments is unknown / variable it is possible to define variadic functions using `*` or `**`. The former is for unnamed arguments which will be treated as a `tuple`, the latter is for named arguments which will be treated as a `dict`.

```
def paste(*x, sep=" "):
    return sep.join(x)
```

```
paste("A")
```

```
'A'
```

```
paste("A","B","C")
```

```
'A B C'
```

```
paste("1","2","3",sep=",")
```

```
'1,2,3'
```

Anonymous functions

Anonymous functions are defined using the `lambda` keyword. They are intended for very short functions (syntactically limited to a single expression, and do not need a return statement)

```
def f(x,y):
    return x**2 + y**2

f(2,3)
```

13

```
type(f)
```

```
<class 'function'>
```

```
g = lambda x, y: x**2 + y**2


g(2,3)
```

13

```
type(g)
```

```
<class 'function'>
```

Function annotations (type hinting)

Python now supports syntax for providing metadata around the expected type of arguments and the return value of a function.

```
def f(x: str, y: str, z: str) -> str:
    return x + y + z
```

These annotations are stored in the `__annotations__` attribute

```
f.__annotations__
```

```
{'x': <class 'str'>, 'y': <class 'str'>, 'z': <class 'str'>, 'return': <class 'str'>}
```

But doesn't actually do anything at runtime

```
f("A","B","C")
```

```
'ABC'
```

```
f(1,2,3)
```

```
6
```

Exercise 2

1. Write a function, `kg_to_lb`, that converts a list of weights in kilograms to a list of weights in pounds (1 kg = 2.20462 lbs). Include a doc string and function annotations.

2. Write a second function, `total_lb`, that calculates the total weight in pounds of an order, the input arguments should be a list of item weights in kilograms and a list of the number of each item ordered.

Classes

Basic syntax

These are the basic components of Python's object oriented system - we've been using them regularly all over the place and will now look at how they are defined and used.

```python
class rect:
  """An object representing a rectangle"""

  # Attributes
  p1 = (0,0)
  p2 = (1,2)

  # Methods
  def area(self):
    return ((self.p1[0] - self.p2[0]) *
            (self.p1[1] - self.p2[1]))

  def set_p1(self, p1):
    self.p1 = p1

  def set_p2(self, p2):
    self.p2 = p2
```

```python
x = rect()
```

```python
x.area()
```

```
2
```

```python
x.set_p2((1,1))
x.area()
```

```
1
```

```python
x.p1
```

```
(0, 0)
```

```python
x.p2
```

```
(1, 1)
```

```python
x.p2 = (0,0)
x.area()
```

```
0
```

Instantiation (constructors)

When instantiating a class object (e.g. `rect()`) we invoke the `__init__()` method if it is present in the classes' definition.

```
class rect:
  """An object representing a rectangle"""

  # Constructor
  def __init__(self, p1 = (0,0), p2 = (1,1)
    self.p1 = p1
    self.p2 = p2

  # Methods
  def area(self):
    return ((self.p1[0] - self.p2[0]) *
            (self.p1[1] - self.p2[1]))

  def set_p1(self, p1):
    self.p1 = p1

  def set_p2(self, p2):
    self.p2 = p2
```

```
x = rect()
x.area()
```

1

```
y = rect((0,0), (3,3))
y.area()
```

9

```
z = rect((-1,-1))
z.p1
```

(-1, -1)

```
z.p2
```

(1, 1)

Method chaining

We've seen a number of objects (e.g., Pandas DataFrames) that allow for method chaining to construct a pipeline of operations. We can achieve the same by having our class methods return itself via `self`.

```python
class rect:
  """An object representing a rectangle"""

  # Constructor
  def __init__(self, p1 = (0,0), p2 = (1,1)
    self.p1 = p1
    self.p2 = p2

  # Methods
  def area(self):
    return ((self.p1[0] - self.p2[0]) *
            (self.p1[1] - self.p2[1]))

  def set_p1(self, p1):
    self.p1 = p1
    return self

  def set_p2(self, p2):
    self.p2 = p2
    return self
```

```python
rect().area()
```

1

```python
rect().set_p1((-1,-1)).area()
```

4

```python
( rect()
  .set_p1((-1,-1))
  .set_p2((2,2))
  .area()
)
```

9

Class object string formating

All class objects have a default print method / string conversion method, but the default behavior is not very useful,

```
print(rect())
```

```
<__main__.rect object at 0x1083e1640>
```

```
str(rect())
```

```
'<__main__.rect object at 0x10e6a3b60>'
```

Both of the above are handled by the `__str__()` method which is implicitly created for our class - we can override this,

```
def rect_str(self):
  return f"Rect[{self.p1}, {self.p2}] => area={self.area()}"

rect.__str__ = rect_str
```

```
rect()
```

```
<__main__.rect object at 0x10b5d9820>
```

```
print(rect())
```

```
Rect[(0, 0), (1, 1)] => area=1
```

```
str(rect())
```

```
'Rect[(0, 0), (1, 1)] => area=1'
```

Class representation

There is another special method which is responsible for printing the object (see `rect()` above) called `__repr__()` which is responsible for printing the classes representation. If possible this is meant to be a valid Python expression capable of recreating the object.

```
def rect_repr(self):
  return f"rect({self.p1}, {self.p2})"

rect.__repr__ = rect_repr
```

```
rect()
```

```
rect((0, 0), (1, 1))
```

```
repr(rect())
```

```
'rect((0, 0), (1, 1))'
```

Inheritance

Part of the object oriented system is that classes can inherit from other classes, meaning they gain access to all of their parent's attributes and methods. We will not go too in depth on this topic beyond showing the basic functionality.

```
class square(rect):
    pass
```

```
square()
```

```
rect((0, 0), (1, 1))
```

```
square().area()
```

```
1
```

```
square().set_p1((-1,-1)).area()
```

```
4
```

Overriding methods

```
class square(rect):
    def __init__(self, p1=(0,0), l=1):
      assert isinstance(l, (float, int)), \
             "l must be a number"

      p2 = (p1[0]+l, p1[1]+l)

      self.l  = l
      super().__init__(p1, p2)

    def set_p1(self, p1):
      self.p1 = p1
      self.p2 = (self.p1[0]+self.l, self.p1[1]+self.l
      return self

    def set_p2(self, p2):
      raise RuntimeError("Squares take l not p2")

    def set_l(self, l):
      assert isinstance(l, (float, int)), \
             "l must be a number"

      self.l  = l
```

```
square()
```

```
square((0, 0), 1)
```

```
square().area()
```

```
1
```

```
square().set_p1((-1,-1)).area()
```

```
1
```

```
square().set_l(2).area()
```

```
4
```

```
square((0,0), (1,1))
```

```
AssertionError: l must be a number
```

```
square().set_l((0,0))
```

```
AssertionError: l must be a number
```

```
square().set_p2((0,0))
```

```
RuntimeError: Squares take l not p2
```

Class attributes

We can examine all of a classes' methods and attributes using `dir()`,

```
[
  dir(rect)
]
```

```
[['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__'
```

Where did `p1` and `p2` go?

```
[
  dir(rect())
]
```

```
[['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__'
```